GUIDE DE SOUTENANCE ORALE & RECETTE

Système Informatisé de Gestion d'Usine de Production • Démonstration & Questions Jury

PROMOTION TWIN_3
Génie Logiciel • 2026

Statut Officiel du Projet : 100% Développé, Testé & Validé sur PostgreSQL 17
Ce document récapitule l'architecture 3-tiers, le scénario chronologique idéal en 6 étapes (15 min) pour la soutenance orale, la traçabilité complète dans PostgreSQL et les réponses aux questions pièges du jury.

1. Architecture 3-Tiers & Stack Technologique Justifiée

Tier Présentation (Frontend React SPA)	Tier Métier & Données (Django REST & PostgreSQL)
• **React 18** + **Vite** + **Tailwind CSS**. • 8 vues métier modulaires conformes au rapport UML. • Écran de connexion conforme au cas *«include» S'authentifier*. • Client HTTP asynchrone avec gestion automatique des tokens JWT.	• **Django 5 & Django REST Framework** (Python 3.12). • **PostgreSQL 17** (17 tables relationnelles du diagramme UML). • Verrouillage de concurrence SQL (select_for_update). • 8 tests unitaires automatisés validés à 100% (8/8 OK).

2. Matrice des 4 Acteurs et Comptes de Démonstration (RBAC)

Rôle (RBAC)	Identifiant	Mot de passe	Périmètre Métier & Habilitations Accordées
ADMINISTRATEUR	`admin`	`admin123`	Supervision globale, gestion des comptes utilisateurs, back-office technique Django.
RESPONSABLE ATELIERS	`resp_atelier`	`resp123`	Création ateliers & lignes, catalogue produits, ordres de fabrication & lots.
COLLABORATEUR RESP.	`collab_resp`	`collab123`	Gestion des équipes, affectation ouvriers, consignation des incidents & temps d'arrêt.
OPÉRATEUR / STOCKS	`operateur1`	`oper123`	Machines, interventions de maintenance avec décrémentation stock, nomenclature (BOM).

3. Démarrage Simplifié en 1 Clic Avant la Soutenance

■ **Double-cliquez sur :** demarrer_projet.bat (à la racine du projet).
Deux fenêtres s'ouvrent : l'API Django sur **http://localhost:8000** et l'application React sur **http://localhost:5173**.
(En cas d'imprévu matériel ou réseau, double-cliquez directement sur standalone_demo.html pour une démo autonome).

SCÉNARIO DE DÉMONSTRATION ORALE (CHRONO 15 MINUTES)

Déroulé pas-à-pas conçu pour captiver le jury et prouver la couverture intégrale du sujet

Étape 1 : Connexion Sécurisée & Cloisonnement RBAC — **0:00 - 2:00**

Ce que vous dites : « Conformément au cas d'utilisation UML obligatoire «include» S'authentifier, l'accès est entièrement verrouillé par jeton JWT. Chaque acteur dispose d'un espace de travail personnalisé selon ses habilitations. »

■ **Action :** Sur **http://localhost:5173**, montrez l'écran de Login officiel. Cliquez sur le badge d'accès rapide **Administrateur** (admin / admin123). Montrez le tableau de bord avec les KPI temps réel (lignes actives, temps d'arrêt cumulé, alertes stock).

Étape 2 : Ateliers, Lignes & Traçabilité des Lots (Modules 1 & 2) — **2:00 - 5:00**

Ce que vous dites : « Le Responsable d'Ateliers gère les capacités industrielles physiques et lance les ordres de fabrication. Chaque lot est tracé avec les ouvriers affectés. »

■ **Action :** Connectez-vous en resp_atelier. Allez sur **Ateliers & Lignes**, cliquez sur + *Créer un Atelier* (ex: Atelier Usinage Lourd #3 avec 4 opérateurs). Allez ensuite sur **Lots de Production** : montrez qu'un lot associe le produit, la date et la liste des **matricules des ouvriers**.

Étape 3 : Équipes d'Ouvriers & Déclaration d'Incident (Modules 4 & 7) — **5:00 - 8:00**

Ce que vous dites : « Le Collaborateur Responsable pilote les équipes humaines et qualifie les aléas de fabrication avec saisie obligatoire du temps d'arrêt machine. »

■ **Action :** Connectez-vous en collab_resp. Allez dans **Équipes & Opérateurs** : montrez l'affectation matricule/équipe. Puis dans **Incidents**, cliquez sur + *Déclarer un Incident* sur le Lot #1001, panne mécanique, **45 min de temps d'arrêt** : montrez le compteur d'arrêt qui s'actualise !

Étape 4 : Maintenance & Décrémentation Atomique de Stock (Module 5) — **8:00 - 11:00 (POINT CLÉ)**

Ce que vous dites : « Voici l'un des algorithmes critiques du sujet : lors d'une intervention de maintenance, la consommation de pièces déclenche une transaction atomique décrémentant immédiatement le stock dans PostgreSQL. »

■ **Action :** Connectez-vous en operateur1. Notez le stock actuel de la pièce *Roulement SKF 6205* (ex: 120 unités). Déclarez une maintenance sur la machine ROBOT-ASS-03, consommez **2 roulements** : montrez que le stock passe instantanément à **118 unités** !

Étape 5 : Nomenclature Récursive (BOM) & Algorithme Anti-Cycle (Module 6) — **11:00 - 13:30**

Ce que vous dites : « Une pièce peut être composée d'autres pièces sur plusieurs niveaux. Notre backend intègre un algorithme de détection de cycle en profondeur (DFS) pour interdire toute boucle récursive infinie. »

■ **Action :** Ouvrez l'onglet **Stock & Nomenclature**. Montrez l'arborescence multi-niveaux du *Rotor 5kW* (arbre, roulements, joints). Modifiez le multiplicateur de calcul pour prouver le dimensionnement dynamique des composants nécessaires.

Étape 6 : Preuve dans PostgreSQL 17 & Banc de Tests Automatisés — **13:30 - 15:00**

Ce que vous dites : « Toutes ces actions sont enregistrées en temps réel dans notre base relationnelle PostgreSQL et couvertes par notre banc de tests unitaires automatisés. »

■ **Action :** Basculez sur **pgAdmin 4**, montrez la table stock_achats_piece avec la valeur 118 à jour. Ouvrez un terminal et exécutez python manage.py test pour prouver les **8/8 tests réussis (OK)** devant le jury.

QUESTIONS DU JURY & ARGUMENTAIRES TECHNIQUES

Les réponses d'ingénierie logicielle précises pour obtenir la note maximale

Q1 : Pourquoi avoir choisi Django REST Framework et PostgreSQL plutôt que du NoSQL ?

Réponse argumentée : « Le modèle industriel d'une usine requiert une forte intégrité référentielle (ACID) : un incident doit obligatoirement être rattaché à un lot unique, une maintenance référence une machine unique. PostgreSQL garantit le respect strict des clés étrangères et la cohérence des écritures concurrentes. Django REST Framework fournit un ORM puissant, un système RBAC natif et un back-office d'administration accélérant la maintenance. »

Q2 : Comment empêchez-vous qu'un stock devienne négatif en cas d'accès simultanés (Race Condition) ?

Réponse argumentée : « Dans l'application maintenance, la méthode consommerPieces() est encapsulée dans un bloc @transaction.atomic avec l'instruction SQL select_for_update(). Cela pose un verrou exclusif sur la ligne de la pièce dans PostgreSQL. Le système vérifie if piece.quantiteStock < quantite: raise ValidationError() avant de décrémenter, garantissant qu'aucune écriture concurrente ne corrompe l'inventaire. »

Q3 : Comment fonctionne l'algorithme anti-cycle dans la nomenclature récursive (BOM) ?

Réponse argumentée : « La nomenclature forme un Graphe Orienté Acyclique (DAG). Dans apps/stock_achats/services.py, avant d'ajouter une relation Parente → Composante, nous exécutons un parcours en profondeur (DFS) depuis la pièce composante. Si la pièce parente est déjà présente dans sa descendance, un cycle infini se formerait. L'algorithme détecte l'anomalie et lève une ValidationError pour rejeter l'opération. »

Q4 : Comment est gérée la sécurité et la communication entre le Frontend et le Backend ?

Réponse argumentée : « L'architecture est une SPA (Single Page Application) découplée communiquant via des requêtes HTTP asynchrones (REST JSON). L'authentification utilise des jetons JWT (SimpleJWT). Lors du login, le backend délivre un access token et un refresh token. Chaque requête suivante porte l'en-tête Authorization: Bearer <token>, inspecté par les permissions Django pour autoriser ou refuser l'accès (403 Forbidden). »

Fiche Mémo des Commandes Clés pour la Soutenance

Action Voulue	Commande Terminal (PowerShell / CMD)	Résultat Immédiat
Lancer l'application complète	`demarrer_projet.bat`	Démarre Django (:8000) et React (:5173).
Prouver les tests unitaires	`cd backend; .\venv\Scripts\python.exe manage.py test`	Exécute les 8 tests sur PostgreSQL (OK).
Ouvrir le Back-office Admin	`URL navigateur : http://localhost:8000/admin/`	Accès super-administrateur aux 17 tables.
Vérifier l'historique Git	`git log --oneline -2`	Affiche les 2 commits de sauvegarde officiels.